МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №3

з дисципліни «Функціональне програмування»

на тему: «Нездатні до змін структури даних (immutability) у Python»

Варіант 6: «Бронювання готелю»

Виконав:
студент групи ФЕП-23
Корж Олексій

Перевірив:
проф. Свелеба С. А.

Львів – 2026

1. Мета роботи

Опанувати використання незмінних структур даних у Python: tuple, frozenset, frozen dataclass, MappingProxyType; замінити прямі мутації на pure updates (повернення нових копій); дослідити гарантії read-only поведінки та порівняти продуктивність immutable-моделі з мутабельною версією.

2. Теоретичні відомості

Незмінність (Immutability): властивість об'єкта, згідно з якою його стан не може бути змінений після створення. Спроба зміни викликає виняток або повертає новий об'єкт.

frozen dataclass: декоратор @dataclass(frozen=True, slots=True) генерує незмінний клас, де будь-яка спроба присвоєння полю генерує FrozenInstanceError.

frozenset: незмінна множина, що підтримує операції об'єднання (|), перетину (&) та є хешованою.

MappingProxyType: обгортка над словником з модуля types, що надає read-only інтерфейс до ключів та значень (спроба модифікації породжує TypeError).

Pure updates: оновлення стану через функцію dataclasses.replace або фабричні методи, що створюють новий екземпляр із заміненими полями, гарантуючи збереження оригіналу.

3. Постановка завдання (Варіант 6)

Тема варіанта 6: Бронювання готелю.

Незмінна модель: Booking, Guest, services (frozenset), meta (MappingProxyType).

Pure-оновлення: add_service (додавання послуги), update_status (зміна статусу), extend_stay (продовження проживання на n ночей).

Перевірки: Захист від мутацій (FrozenInstanceError, TypeError), стабільність тестів, хешованість сутностей та заміри швидкодії.

4. Хід роботи та лістинг програми

Незмінна доменна модель (model.py)

```
from __future__ import annotations

from collections.abc import Iterable
from dataclasses import dataclass
from types import MappingProxyType
from typing import Any


@dataclass(frozen=True, slots=True)
class Guest:
    name: str
    email: str
    phone: str


@dataclass(frozen=True, slots=True)
class BookingKey:
    booking_id: int
    room_number: int


@dataclass(frozen=True, slots=True)
class Booking:
    booking_id: int
    guest: Guest
    room_number: int
    nights: int
    price_per_night: float
    status: str
    services: frozenset[str]
    meta: MappingProxyType[str, Any]

    @classmethod
    def create(
        cls,
        booking_id: int,
        guest: Guest,
        room_number: int,
        nights: int,
        price_per_night: float,
        status: str = "pending",
        services: Iterable[str] | None = None,
        meta: dict[str, Any] | None = None,
    ) -> Booking:
        frozen_services = frozenset(services) if services else frozenset()
        proxy_meta = MappingProxyType(dict(meta)) if meta else MappingProxyType({})
        return cls(
            booking_id=booking_id,
            guest=guest,
            room_number=room_number,
            nights=nights,
            price_per_night=price_per_night,
```

```
            status=status,
            services=frozen_services,
            meta=proxy_meta,
        )

    @property
    def key(self) -> BookingKey:
        return BookingKey(self.booking_id, self.room_number)
```

Мутабельна модель (для порівняння) (mutable_model.py)

```
from __future__ import annotations

from typing import Any


class MutableGuest:
    def __init__(self, name: str, email: str, phone: str) -> None:
        self.name = name
        self.email = email
        self.phone = phone


class MutableBooking:
    def __init__(
        self,
        booking_id: int,
        guest: MutableGuest,
        room_number: int,
        nights: int,
        price_per_night: float,
        status: str,
        services: list[str],
        meta: dict[str, Any],
    ) -> None:
        self.booking_id = booking_id
        self.guest = guest
        self.room_number = room_number
        self.nights = nights
        self.price_per_night = price_per_night
        self.status = status
        self.services = services
        self.meta = meta

    def add_service(self, service: str) -> None:
        self.services.append(service)

    def update_status(self, new_status: str) -> None:
        self.status = new_status

    def extend_stay(self, additional_nights: int) -> None:
        self.nights += additional_nights
```

Чисті функції оновлення (core.py)

```
from __future__ import annotations
```

```
from dataclasses import replace
from types import MappingProxyType
from typing import Any

from model import Booking


def add_service(booking: Booking, service: str) -> Booking:
    new_services = booking.services | {service}
    return replace(booking, services=new_services)


def update_status(booking: Booking, new_status: str) -> Booking:
    return replace(booking, status=new_status)


def extend_stay(booking: Booking, additional_nights: int) -> Booking:
    if additional_nights < 0:
        raise ValueError("additional_nights must be non-negative")
    return replace(booking, nights=booking.nights + additional_nights)


def update_meta(booking: Booking, key: str, value: Any) -> Booking:
    current = dict(booking.meta)
    current[key] = value
    return replace(booking, meta=MappingProxyType(current))


def calculate_total(
    booking: Booking, service_prices: dict[str, float] | None = None
) -> float:
    base = booking.nights * booking.price_per_night
    if not service_prices:
        return round(base, 2)
    services_cost = sum(service_prices.get(srv, 0.0) for srv in booking.services)
    return round(base + services_cost, 2)
```

Демонстраційна програма (main.py)

```
from __future__ import annotations

from core import add_service, calculate_total, extend_stay, update_status
from model import Booking, Guest


def main() -> None:
    guest = Guest(
        name="Олексій Корж",
        email="oleksiy@example.com",
        phone="+380501234567",
    )
    booking = Booking.create(
        booking_id=1001,
        guest=guest,
        room_number=305,
        nights=3,
        price_per_night=1200.0,
        status="confirmed",
        services=["breakfast"],
```

```
        meta={"source": "website", "vip": True},
    )

    print("Початкове бронювання:")
    print(f"ID: {booking.booking_id}, Гість: {booking.guest.name}")
    print(f"Ночей: {booking.nights}, Статус: {booking.status}")
    print(f"Послуги: {set(booking.services)}")

    prices = {"breakfast": 200.0, "spa": 500.0, "parking": 150.0}
    print(f"Базова вартість: {calculate_total(booking, prices)} грн")

    b_with_spa = add_service(booking, "spa")
    b_extended = extend_stay(b_with_spa, 2)
    b_final = update_status(b_extended, "checked_in")

    print("\nОновлене бронювання (нова копія):")
    print(f"Ночей: {b_final.nights}, Статус: {b_final.status}")
    print(f"Послуги: {set(b_final.services)}")
    print(f"Підсумкова вартість: {calculate_total(b_final, prices)} грн")

    print(
        f"\nНезмінність оригіналу: ночей={booking.nights}, "
        f"послуг={len(booking.services)}"
    )


if __name__ == "__main__":
    main()
```

Тести незмінності та захисту (tests/test_immutability.py)

```
from dataclasses import FrozenInstanceError

import pytest
from core import add_service, calculate_total, extend_stay, update_meta, update_status
from model import Booking, BookingKey, Guest


def test_frozen_attributes_cannot_be_mutated(sample_booking: Booking) -> None:
    with pytest.raises(FrozenInstanceError):
        sample_booking.status = "confirmed"  # type: ignore[misc]

    with pytest.raises(FrozenInstanceError):
        sample_booking.nights = 5  # type: ignore[misc]

    with pytest.raises(FrozenInstanceError):
        sample_booking.guest.name = "New Name"  # type: ignore[misc]


def test_mapping_proxy_read_only(sample_booking: Booking) -> None:
    with pytest.raises(TypeError):
        sample_booking.meta["vip"] = True  # type: ignore[index]


def test_add_service_is_pure(sample_booking: Booking) -> None:
    updated = add_service(sample_booking, "breakfast")
    assert "breakfast" not in sample_booking.services
    assert "breakfast" in updated.services
    assert updated.booking_id == sample_booking.booking_id
```

```
    assert updated is not sample_booking


def test_update_status_is_pure(sample_booking: Booking) -> None:
    updated = update_status(sample_booking, "confirmed")
    assert sample_booking.status == "pending"
    assert updated.status == "confirmed"
    assert updated is not sample_booking


def test_extend_stay_is_pure(sample_booking: Booking) -> None:
    updated = extend_stay(sample_booking, 3)
    assert sample_booking.nights == 2
    assert updated.nights == 5
    assert updated is not sample_booking


def test_extend_stay_negative_error(sample_booking: Booking) -> None:
    with pytest.raises(ValueError):
        extend_stay(sample_booking, -1)


def test_update_meta_is_pure(sample_booking: Booking) -> None:
    updated = update_meta(sample_booking, "discount", 0.15)
    assert "discount" not in sample_booking.meta
    assert updated.meta["discount"] == 0.15
    assert updated is not sample_booking


def test_calculate_total(sample_booking: Booking) -> None:
    prices = {"wifi": 50.0, "spa": 300.0}
    total = calculate_total(sample_booking, prices)
    assert total == (2 * 1000.0 + 50.0)


def test_hashability_and_keys(sample_guest: Guest, sample_booking: Booking) -> None:
    guest_set = {sample_guest}
    assert sample_guest in guest_set

    key1 = sample_booking.key
    key2 = BookingKey(sample_booking.booking_id, sample_booking.room_number)
    assert hash(key1) == hash(key2)
    booking_dict = {key1: sample_booking}
    assert booking_dict[key2] == sample_booking

    with pytest.raises(TypeError):
        hash(sample_booking)
```

Тести продуктивності (tests/test_performance.py)

```
import time

from core import add_service, extend_stay
from model import Booking, Guest
from mutable_model import MutableBooking, MutableGuest


def test_performance_benchmark() -> None:
    iterations = 5000
```

```
    guest = Guest("Test", "t@example.com", "123")
    booking = Booking.create(1, guest, 101, 1, 100.0)

    start_immutable = time.perf_counter()
    curr_b = booking
    for i in range(iterations):
        curr_b = extend_stay(curr_b, 1)
        curr_b = add_service(curr_b, f"srv_{i % 10}")
    time_immutable = time.perf_counter() - start_immutable

    m_guest = MutableGuest("Test", "t@example.com", "123")
    m_booking = MutableBooking(1, m_guest, 101, 1, 100.0, "pending", [], {})

    start_mutable = time.perf_counter()
    for i in range(iterations):
        m_booking.extend_stay(1)
        m_booking.add_service(f"srv_{i % 10}")
    time_mutable = time.perf_counter() - start_mutable

    assert time_immutable > 0
    assert time_mutable > 0
    assert curr_b.nights == 1 + iterations
    assert m_booking.nights == 1 + iterations
```

5. Результати виконання та тестування

Запуск демонстраційної програми main.py

```
$ python main.py
Початкове бронювання:
ID: 1001, Гість: Олексій Корж
Ночей: 3, Статус: confirmed
Послуги: {'breakfast'}
Базова вартість: 3800.0 грн

Оновлене бронювання (нова копія):
Ночей: 5, Статус: checked_in
Послуги: {'breakfast', 'spa'}
Підсумкова вартість: 6700.0 грн

Незмінність оригіналу: ночей=3, послуг=1
```

Результати виконання тестів pytest

```
$ pytest -q
..........                                                    [100%]
10 passed in 0.57s
```

Статична перевірка типів mypy --strict

```
$ mypy --strict .
Success: no issues found in 7 source files
```

Перевірка форматування ruff та black

```
$ ruff check .
All checks passed!

$ black --check .
All done! ✨ 🍰 ✨
7 files would be left unchanged.
```

6. Висновки

У лабораторній роботі №3 реалізовано повноцінну незмінну модель бронювання готелю. Використання frozen dataclass, frozenset та MappingProxyType повністю унеможливило небажані побічні ефекти та мутації стану. Заміри продуктивності підтвердили, що створення нових копій через dataclasses.replace виконується за мікросекунди, забезпечуючи високу швидкодію та надійність.